DOBOT / EXECUTION GATEWAY

原子技能执行网关

接口与联调手册

文档编号 DBT-GW-INT-001

文档版本 1.1

发布日期 2026-10-12

适用配置 dobot-dag/1

实现基线 3eed0a9

适用于上游编排系统与 Dobot X-Trainer 双 Nova 设备侧集成。
包含接口约定、运行包示例、执行管理、联调流程与验收要求。

目录与阅读说明

01 系统范围与职责 3

02 接入约定与能力目录 4

03 接口索引 5

04 运行包与图约束 6

05 单臂运行包示例 7

06 双臂依赖示例 8

07 提交、预检与幂等 9

08 状态、事件与异常 10

09 Mock 联调流程 11

10 真机部署与现场复位 12

11 上游连接与事件回传 13

12 联调验收与维护 14

接入开发先阅读第 02 至 08 章；联调人员按第 09 至 12 章执行。所有命令在仓库根目录运行；报文示例中的任务标识和令牌为占位值。

修订说明

1.1：规范角色与术语，重组接口和验收内容，补充完整单臂示例；更新字体、表格、代码与图示排版。接口实现及机器人控制行为未变更。

01 系统范围与职责

本手册规定 Dobot X-Trainer 双 Nova 原子技能执行网关的接入方式、运行包格式、执行管理与联调验收要求。适用于上游编排系统开发人员、设备侧集成人员和现场操作员。

上游系统提交技能级执行图；执行网关校验运行包，将注册技能展开为设备动作，保存执行状态并返回节点事件。接口不接收任意关节角、笛卡尔坐标或厂商控制指令。



运行包提交与校验 → 技能展开与执行 → 状态查询与事件回传

1.1 责任划分

角色	职责
上游编排系统	读取能力目录；生成运行包；管理任务与请求标识；消费状态及事件；决策失败后的恢复流程。
执行网关	校验版本、图结构、参数及场景；管理设备独占与幂等；执行技能；记录状态、事件与停止结果。
现场操作员	确认硬件状态、示教位置及初始场景；执行现场复位确认；监护真机测试；处置异常。

1.2 适用版本

项目	版本或标识
外部运行包	RuntimeExecutionPackage；schema_version = 2.1
执行图	graph_version = 0.1
支持子集	dobot-dag/1
技能	embodied.pick / embodied.place；skill_version = 1.0.0

本手册描述已实现的受限接口。未列入支持范围的动态图、条件节点、恢复节点和断点续跑不可提交。能力扩展应先完成接口约定、实现与联调验证。

> 结果判定：HTTP 受理、Mock 执行成功和控制器执行记录分别对应不同证据。物体抓取与放置结果须由现场确认或独立传感器验收。

02 接入约定与能力目录

2.1 HTTP 约定

项目	约定
默认地址	http://127.0.0.1:8080；实际地址由部署配置确定。
编码	请求与响应均为 UTF-8 JSON。
POST 请求	Content-Type: application/json；必须携带一个 Content-Length。
请求大小	1 字节至 2 MiB；不接受 chunked 请求体。
鉴权	服务配置令牌时，所有请求携带 Authorization: Bearer <token>。
数据有效性	JSON 顶层为 object；拒绝重复字段和非有限数值。

默认回环地址用于本机联调。非回环监听要求令牌及 TLS；实验环境若采用明文 HTTP，须显式启用 `--allow-insecure-http`。令牌通过环境变量注入，不写入运行包或仓库。

2.2 读取能力目录

```
GET /v1/catalog
Authorization: Bearer <token>
```

能力目录包含 `catalog_revision`、`devices`、`atomic_skills` 和 `execution_profile`。目录响应自身的 `schema_version` 为 1.0，与运行包版本 2.1 是不同字段。

上游系统应按目录选择设备、技能和版本，将返回的 `catalog_revision` 原样写入运行包。网关依据场景、模式和技能版本生成目录修订标识；Mock 与真机目录不可互换。

2.3 健康与就绪

`GET /health` 返回 `status`、`mode`、`catalog_revision`、`active_execution_id` 和 `quarantined`。`status=ok` 表示 HTTP 服务可响应，不表示真机已就绪。真机就绪检查、设备独占与场景一致性在执行链路中继续验证。

> 目录过期：收到 STALE_CATALOG_REVISION 后，重新读取目录并由上游重新编译运行包。不得仅替换旧包中的目录标识后重发。

03 接口索引

以下路径均相对于网关基础地址。执行提交为异步接口，HTTP 202 只表示受理。

方法	路径	成功响应
GET	/health	200；服务状态
GET	/v1/catalog	200；能力目录
POST	/v1/execute-graph	202；执行标识
POST	/device-gateway/v1/execution-plans/validate	200；预检及预留结果
POST	/device-gateway/v1/executions	202；执行标识
GET	/v1/executions/{id}	200；执行状态
GET	/v1/executions/{id}/events	200；分页事件
POST	/v1/executions/{id}/cancel	200；取消处理结果
GET	/v1/delivery-status	200；回传队列状态

3.1 兼容路径

- `/v1/capabilities` 是 `/v1/catalog` 的别名。
- 执行状态、事件与取消也支持 `/device-gateway/v1/executions/{id}` 前缀。
- `/v1/integrations/execution-graphs/execute` 属于上游系统接口，不是本执行网关路由。

3.2 请求封装

接口	JSON 请求体
单次提交	graph：完整运行包。
两阶段预检	package：完整运行包；reserve_resources：布尔值，默认 true。
两阶段执行	package：预检时的同一运行包；validation_token：预检返回的令牌。
取消	空 object，即 {}。

预检和执行提交均要求请求级幂等键。请求头 `Idempotency-Key` 和请求体 `request_id` 至少提供一项；同时提供时必须相同。取消接口不要求该幂等键。

3.3 常见 HTTP 错误

400 表示请求形状或 JSON 错误；401 表示鉴权失败；404 表示路由或执行标识不存在；409 表示幂等、资源、令牌或场景冲突；413 表示体积越界；415 表示 Content-Type 错误；422 表示合同校验失败；500 表示内部异常；502 表示上游调用失败；503 表示网关关闭中。

04 运行包与图约束

4.1 核心字段

字段	要求
schema_version / graph_version	固定为字符串 2.1 / 0.1。
package_id	必填非空标识，最长 200 字符；用作图标识。
experiment_id	必填非空实验标识，最长 200 字符。
catalog_revision	必须与当前能力目录一致。
entry_node_ids	必须准确列出所有无前驱节点。
nodes	1 至 100 个节点；node_id 唯一。
edges	每条包含 source、target、edge_type；只支持 success。

`run_id`、`recipe_id` 和 `version` 可用于业务追溯；run_id 建议由上游统一生成。若提供 `graph_id`，必须等于 `package_id`。单根图也支持 `entry_node_id`；多根图使用 `entry_node_ids`。支持子集由目录的 `execution_profile` 声明。

4.2 节点与参数

节点	结构
action	status=pending；skill_invocation 包含 skill_id、skill_version、device_id、idempotency_key、parameters。
wait	status=pending；duration_s 为 0 至 60 秒；不得携带 skill_invocation。
公共超时	timeout_s 为 0.01 至 900 秒，默认 180 秒；包含技能内部执行及资源等待。

抓取参数为 `object_id`、`source_ref`；放置参数为 `object_id`、`destination_ref`。左设备绑定 A，引用为 `object:A` 或 `target:target_A`；右设备绑定 B，引用为 `object:B` 或 `target:target_B`。

4.3 执行语义

- 图必须无环；同臂动作必须有明确依赖，不能依靠数组顺序。
- 多条入边为 AND：全部前驱成功后才启动节点。
- 抓取引用须符合当前逻辑位置；放置须符合持物状态；图结束须释放已抓物体。
- 每个装置同时只允许一个执行；Mock 可跨臂并行，真机共享工作区保守串行。

不支持 condition、observe、parallel、human_approval、recovery 节点、非 success 边、自定义条件、任意坐标、恢复图或断点续跑。并行通过多个根节点与依赖边表达。`on_fail` 可保存外部决策元数据，不触发本地自动重试。

05 单臂运行包示例

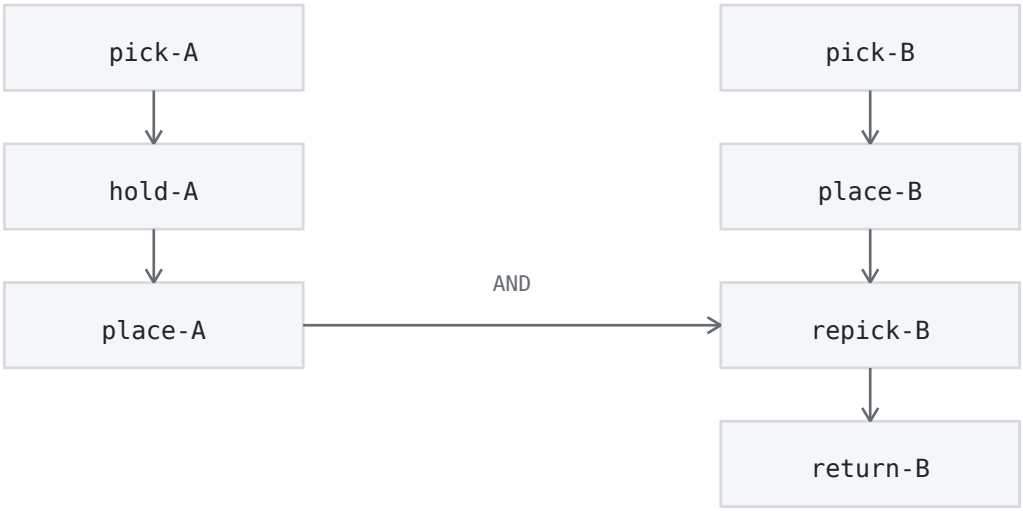
下例描述左臂抓取 A 并放置到目标 A。`catalog_revision` 必须来自当前目录；任务与幂等标识由调用方生成。完整可复用文件见 `examples/runtime-single-arm.json`。

```
{
  "schema_version": "2.1",
  "graph_version": "0.1",
  "package_id": "runtime-single-001",
  "run_id": "single-001",
  "experiment_id": "experiment-001",
  "catalog_revision": "CATALOG_REVISION_FROM_GATEWAY",
  "entry_node_ids": ["pick-A"],
  "nodes": [
    {
      "node_id": "pick-A", "node_type": "action",
      "status": "pending", "timeout_s": 180,
      "skill_invocation": {
        "skill_id": "embodied.pick",
        "skill_version": "1.0.0",
        "device_id": "dobot-nova-left",
        "idempotency_key": "single-001:pick-A",
        "parameters": {
          "object_id": "A", "source_ref": "object:A"
        }
      }
    },
    {
      "node_id": "place-A", "node_type": "action",
      "status": "pending", "timeout_s": 180,
      "skill_invocation": {
        "skill_id": "embodied.place",
        "skill_version": "1.0.0",
        "device_id": "dobot-nova-left",
        "idempotency_key": "single-001:place-A",
        "parameters": {
          "object_id": "A",
          "destination_ref": "target:target_A"
        }
      }
    }
  ],
  "edges": [{
    "source": "pick-A", "target": "place-A",
    "edge_type": "success"
  }]
}
```

重复投递同一次请求时保留原包及原键；重新规划的任务使用新包和新键，同时依据最新物体状态确定引用。不得将已完成图换键后直接重放。

06 双臂依赖示例

七节点示例用于验证跨臂分支、定时等待与 AND 汇合。完整运行包见 `examples/runtime-bimanual-hold-return.json`；图中每条边均为 success。



6.1 节点定义

节点	操作	直接前驱
pick-A	左臂从 object:A 抓取 A	无
pick-B	右臂从 object:B 抓取 B	无
hold-A	等待 5 秒，保持 A 持物状态	pick-A
place-A	左臂将 A 放至 target:target_A	hold-A
place-B	右臂将 B 放至 target:target_B	pick-B
repick-B	右臂从 target:target_B 再抓 B	place-A AND place-B
return-B	右臂将 B 放回 object:B	repick-B

6.2 验证要点

两个根节点为 `pick-A` 和 `pick-B`。`repick-B` 必须等待两次放置都成功；该约束由依赖边保证，不能用固定延时替代。

`hold-A` 在 `pick-A` 成功后计时。`place-A` 无须等待 `place-B`，两个分支通过后续汇合约束返程。Mock 中可观察跨臂逻辑并行；真机网关对共享工作区动作使用串行互斥。

示例结束的逻辑位置为 A 在目标 A、B 在初始位置。重复演示应使用新的 Mock 数据目录，或关闭网关后记录 Mock 场景复位。

07 提交、预检与幂等

7.1 两阶段提交

上游先预检并预留资源，再使用返回令牌提交同一包。默认预留有效期为 60 秒，以 `expires_at_unix` 为准。以下请求体中的 package 表示完整运行包。

```
POST /device-gateway/v1/execution-plans/validate
Content-Type: application/json
Idempotency-Key: preflight-single-001
Authorization: Bearer <token>
```

请求体包含 `package` 与 `reserve_resources: true`。成功响应返回 `accepted`、`validation_token`、`catalog_revision`、`expires_at_unix`、`reservations`、`diagnostics` 和 `package`。

```
POST /device-gateway/v1/executions
Content-Type: application/json
Idempotency-Key: dispatch-single-001
Authorization: Bearer <token>
```

执行请求体包含原 `package` 与 `validation_token`。令牌绑定包内容，不能更换包；过期或预留丢失后，应使用新的预检键重新预检。`reserve_resources: false` 只做检查，返回空令牌，不能据此使用两阶段执行接口。

7.2 单次提交

向 `POST /v1/execute-graph` 提交以 `graph` 为字段的完整运行包，并携带独立执行幂等键。已有预留时须走两阶段接口，单次入口不能绕过资源预留。

HTTP 202 响应结构如下；标识和时间值为示例。

```
{
  "execution_id": "execution-example",
  "status": "accepted",
  "graph_id": "runtime-single-001",
  "accepted_at": "2026-10-12T00:00:00+00:00"
}
```

7.3 两级幂等

请求级键区分预检与执行提交；相同键及同一包返回原结果，换包返回 `IDEMPOTENCY_CONFLICT`。节点级 `skill_invocation.idempotency_key` 在运行包内唯一，且不可被另一个执行重复使用。网络超时后先查询原执行，或保留原键重试；网关不会自动重放重启时未完成的物理动作。

08 状态、事件与异常

8.1 执行状态

`GET /v1/executions/{id}` 返回 `execution_id`、`status`、`mode`、`node_statuses`、`error`、`modeled_object_state` 和 `verification` 等字段。执行期间可出现 accepted、running、finalizing、cancelling；终态为 succeeded、failed、cancelled 或 interrupted。

`succeeded` 表示技能执行流程完成；`verification.level=command_trace` 不独立确认物体物理状态。取消请求可在停止收尾期间返回 cancelling，须继续查询终态和错误结果。

8.2 事件消费

```
GET /v1/executions/{id}/events?after=0&limit=100
```

响应为 `{"events": [...]}`。`after` 是已消费的 sequence，返回更大的序列号；`limit` 为 1 至 1000，默认 100。按 execution_id 与 sequence 分页推进，按 event_id 去重。事件保留上游 node_id，调用方不依赖内部底层节点编号。

8.3 取消与错误结构

```
POST /v1/executions/{id}/cancel
Content-Type: application/json

{}
```

合同错误响应包含 `code`、`message`、`path`、`severity`、`retryable` 和 `details`。当前合同错误的 retryable 为 false；恢复策略由上游结合错误类型决策。

错误码	处理
STALE_CATALOG_REVISION	重新读取目录并重新编译。
IDEMPOTENCY_CONFLICT	核对包和请求键，停止盲目重试。
RESOURCE_CONFLICT	查询现有执行及控制所有者。
VALIDATION_TOKEN_EXPIRED	使用新预检键重新预检。
VALIDATION_TOKEN_MISMATCH	核对预检与提交包是否一致。
SCENE_RESET_REQUIRED	现场确认初始场景并记录复位。

真机停止未确认或运行未完整结束时保持隔离。保留执行证据，先确认设备与物体状态，再处理恢复；不得以删除锁或重复提交代替现场处置。

09 Mock 联调流程

Mock 网关使用真实 HTTP 与持久化存储，设备动作由模拟适配器完成，不导入硬件控制链。Python 3.10 或更新版本即可运行。下列命令均在仓库根目录执行。

9.1 执行网关侧

在终端一启动本机 Mock 网关。示例使用独立数据目录，避免与真机或已有演示共用状态。

```
python -B -m task_skills.run_gateway `
  --mode mock --host 127.0.0.1 --port 8080 `
  --data-dir ./gateway-data/mock-session-01
```

9.2 上游系统侧

在终端二使用模拟客户端验证目录读取、编包、两阶段提交、状态与事件查询。

```
python -B -m task_skills.gateway.demo_client `
  --url http://127.0.0.1:8080 --two-phase `
  --output ./gateway-data/demo-request.json
```

省略 `--two-phase` 可验证单次提交。模拟客户端仅允许 Mock 网关，不作为真机下发工具。内置演示的等待为 1 秒；需要验证 5 秒等待时使用对应完整运行包。

预期结果为 `status=succeeded`，七个节点均成功，事件共 16 条。结果属于模拟执行证据，不代表机械臂或上游真实系统已经验收。

9.3 重复演示与自动烟测

同一数据目录保存逻辑物体状态和幂等历史。再次运行初始场景示例前，关闭网关并记录 Mock 复位，或选择新目录。

```
python -B -m task_skills.gateway.operator `
  --mode mock --data-dir ./gateway-data/mock-session-01 `
  --confirm RESET_VERIFIED
```

自动烟测分别启动临时 Mock 服务验证两种提交方式，随后退出。

```
python -B tests/gateway_cli_smoke.py `
  --output ./gateway-data/cli-evidence
```

10 真机部署与现场复位

真机模式仅支持 Windows 硬件环境。执行网关启动不会自动清报警、使能或移动机械臂。部署前完成驱动、网络、串口、示教场景和物理绑定核验。

10.1 执行前条件

- 两臂位于示教 home，已使能、空闲且无报警；夹爪张开。
- A/B 处于各自初始位置，目标区域无占用；标定与装置匹配。
- 旧 CLI、ZMQ、遥操作和网关控制进程已停止；同一装置只有一个控制所有者。
- 所有代码副本共用同一设备锁路径；现场急停与监护条件已落实。

本装置夹爪方向为 0 闭合、255 张开，且物理夹爪绑定与驱动命名不同。迁移装置须实测，不得直接复用示教或默认方向。

10.2 记录复位并启动

令牌应已通过受控环境配置注入。示例使用回环地址，跨机器访问须配置 TLS 与相应监听地址。

```
$env:DOBOT_DEVICE_LOCK_PATH = 'C:/DobotRuntime/device-owner'

python -B -m task_skills.gateway.operator `
  --mode live --data-dir C:/DobotRuntime/gateway-data/live `
  --confirm RESET_VERIFIED

python -B -m task_skills.run_gateway `
  --mode live --allow-live --host 127.0.0.1 --port 8080 `
  --data-dir C:/DobotRuntime/gateway-data/live
```

`DOBOT_GATEWAY_TOKEN` 为真机必需环境变量。跨机器部署可使用 `--host 0.0.0.0 --tls-cert <cert> --tls-key <key>`。operator 仅记录已确认的场景，不发送硬件指令，不负责使能，也不删除控制所有者锁。

10.3 异常恢复顺序

1. 停止控制进程，现场确认两臂与夹爪静止。

2. 处理未完成任务，恢复物体、夹爪与示教 home 状态。

3. 核实 owner.json 对应进程已退出后，由现场管理员处置残留所有者目录。

4. 对同一真机数据目录记录现场复位，再启动网关并提交新的完整任务。

数据目录固定 Mock / live 模式，不能混用。复位不会删除历史；更换数据目录不应绕过共享场景状态和设备独占约束。

11 上游连接与事件回传

未配置上游地址时，网关仍保存本地事件并提供查询，但不会向外投递。启用连接时，基础 URL 只由服务端部署配置指定，请求体不能提供任意回调地址。

11.1 连接配置

```
python -B -m task_skills.run_gateway `
  --mode mock --compiler-url https://planner.example.org `
  --data-dir ./gateway-data/partner-mock
```

上游鉴权令牌通过 `DOBOT_COMPILER_TOKEN` 注入。该令牌与调用本网关的 `DOBOT_GATEWAY_TOKEN` 分别管理。

11.2 网关调用的上游路径

方法	路径	用途
POST	/v1/integrations/catalog/validate	校验能力目录
POST	/v1/integrations/catalog/snapshots	发布目录快照
POST	/v1/integrations/device-state-events	设备状态事件
GET	/v1/integrations/execution-plans/{run_id}	按运行标识拉取运行包
POST	/v1/executions/{execution_id}/events	执行事件回传

目录校验通过后才发布快照。拉包为可选扩展：配置上游后，提交请求可用 `run_id` 获取完整包或以 `package` 字段封装的包，随后执行相同校验流程。

11.3 投递与 ACK

事件与执行状态事务落盘，outbox 按流保序重试；同一事件重试保持 event_id 与 sequence。网络失败、404 或响应解析错误保留待投递记录。服务重启继续投递已有事件，未完成的物理动作不会自动重放。

当前 ACK 为 2xx JSON object；204 空响应按空 object 处理。若返回 accepted，必须为布尔 true；响应只要包含 code 字段即不视为确认。diagnostics 如存在须为 object 列表，且无 severity=blocking 项。该约定应在真实上游联调时固定。

`GET /v1/delivery-status` 用于检查待投递、已确认及错误记录，并通过 `upstream_configured` 区分未配置与投递成功。

12 联调验收与维护

联调按合同、Mock、真机三阶段推进。每阶段保存运行包、目录修订、执行标识、节点状态、事件与错误记录；避免只保存终态截图。

检查项	验收要求
合同与版本	双方使用同一目录修订、设备与技能版本；合法包可校验。
拒绝行为	环、未知技能、不支持节点或边、错绑对象及过期目录被稳定拒绝。
单次与两阶段	两条提交链路均可追溯到执行标识与节点终态。
幂等	重试原键及原包不重复执行；换包和节点键复用可识别。
依赖	双臂示例的返程抓取等待两次放置均成功。
取消与重启	能查询停止收尾及终态；未知状态隔离；未完成动作不自动重放。
回传	event_id 可去重，sequence 有序；断网后恢复投递。
真机结果	低速现场验证，控制记录与物体实际结果分别验收。

12.1 离线测试

```
python -B -m unittest discover -s tests `
  -p test_execution_gateway.py -q
python -B -m unittest discover -s tests `
  -p test_atomic_runner_unittest.py -q
python -B -m unittest discover -s tests `
  -p test_teleop_regressions.py -q
```

遥操作回归需要 NumPy，应使用项目虚拟环境或已有机器人环境。网关与原子执行测试可在标准库环境运行。

12.2 参考文件与版本维护

- `runtime-package.profile.schema.json`：结构约束；目录与逻辑场景约束仍由网关校验。
- `runtime-package.example.json`：默认 1 秒等待的七节点演示。
- `examples/runtime-single-arm.json`：单臂示例。
- `examples/runtime-bimanual-hold-return.json`：5 秒保持与双臂返程示例。
- `manual/build_manual.py` 与手册 Markdown：排版生成器及完整正文。

接口变更应同时更新代码、Schema、示例与手册，并重新验证合法包、拒绝行为和提交链路。当前网关真实上游合同联调及真机端到端验收尚未完成；力觉、掉落检测、重复成功率与实时避碰不属于本版本已验收能力。